MailGuard

Email Threat & Forensics Console

Project Guide — How It Works, How To Run It, and Suggested Improvements

Prototype build with an integrated NLP composite scoring engine (DistilBERT transformer + TF-IDF/Logistic Regression), prepared for a college hackathon demo.

1. What This Project Does

MailGuard is an analyst-facing web application that ingests raw **.eml** email files and runs them through a five-step explainable threat-detection pipeline, then shows a security analyst a fused fraud score and a click-through 'flowchart' of exactly what was checked and what was found at each step — the same way a human analyst would manually triage a suspicious email, except automated and instant.

The five pipeline steps

1. **Ingestion & Integrity Hashing** — parses the .eml file, computes a SHA-256 hash for chain-of-custody.
2. **Header Authentication** — checks SPF/DKIM/DMARC alignment and the From/Reply-To/Return-Path relationship.
3. **Origin Trace** — geolocates the true sending IP from the Received chain, flags VPN/proxy/hosting ranges.
4. **NLP Semantics & Social Engineering Analysis** — *this is the step you asked to integrate models into.* Scores the email body/subject for phishing/social-engineering language.
5. **Link & Attachment Sandbox** — checks anchor-text/href mismatches, typosquat domains, and attachment hashes.

A **composite scoring engine** (already present in the codebase, `fusion.py`) combines all four scored steps into one 0-100 fraud score using fixed weights (Auth 35% / Origin 20% / Content 25% / Links 20%), with a hard-fail override (e.g. a malicious attachment) that forces the score to 100 regardless of the weighted sum.

2. What Was Integrated For This Build

The zip you provided already had the full application scaffolded (FastAPI backend, React frontend, SQLite database, the 5-step pipeline, the fusion engine). The piece that was still a placeholder was step 4 — the NLP content module only used curated keyword/regex matching. Your notebook (`NLP and LR.ipynb`) had already built and validated two real trained models plus a composite scoring function for exactly this purpose. This build wires those into the existing app.

2.1 New file: backend/app/modules/nlp_engine.py

Houses both models from your notebook as reusable, lazily-loaded singletons (loaded/trained once per server process, not per request):

- **Transformer** — `cybersectony/phishing-email-detection-distilbert_v2.4.1`, downloaded from Hugging Face on first run. Toggle with `USE_TRANSFORMER_MODEL` in .env.
- **TF-IDF + Logistic Regression** — trained on `backend/data/email_text.csv` (53,672 labelled rows) on first startup, then cached to `backend/app/model_cache/` (via joblib) so restarts are instant.
- `combine_scores()` — your composite engine, unchanged weights (0.45 transformer / 0.55 TF-IDF).

- `get_tfidf_explanation()` — returns the top words in *this specific email* that pushed the Logistic Regression score toward phishing, ported directly from your notebook.

2.2 Rewritten: backend/app/modules/content_module.py

This module now:

- Calls `nlp_engine.analyze_text()` to get both model scores and the composite score.
- Still runs the original urgency / financial-diversion / authority-impersonation regex scan — kept as supporting evidence and as a safety net if the ML models are ever unavailable.
- Final content-step score = **65% ML composite score + 35% regex cue score** (0-100 scale).
- Builds a new `flag_summary` field: a plain-English paragraph naming exactly which model flagged the email, its confidence, the top suspicious words, and any matched social-engineering phrases.

Why blend instead of fully replacing the regex layer? The regex cues are 100% explainable and free (no model dependency), while the ML layer generalizes far better to real-world phrasing. Blending means the step degrades gracefully — even with both ML models switched off, the step still works from regex alone.

2.3 Frontend: 'Why this was flagged' panel

In `frontend/src/components/FlowChart.jsx`, clicking the 'NLP Semantics & Social Engineering Analysis' box now shows a highlighted callout with the `flag_summary` text before the raw evidence table — this is the answer to 'generate a proper summary of why an account has been flagged' when the analyst clicks into that step. Styling added in `styles.css` (`.nlp-flag-summary`).

2.4 Other changes

- `backend/data/email_text.csv` — your dataset, bundled with the project.
- `backend/sample_emails/phishing_sample1.eml` — the 'Spot Admissions' email, renamed as requested.
- `backend/sample_emails/legit_sample1.eml` — the 'Two new free courses' email, renamed as requested.
- `backend/app/config.py` — added `USE_TRANSFORMER_MODEL` setting.
- `backend/requirements.txt` — added pandas, scikit-learn, joblib, numpy, transformers, torch.
- `backend/app/main.py` — added a `GET /health/nlp` endpoint that reports whether the TF-IDF model trained and whether the transformer loaded — check this first after startup.

3. How The Scoring Works, End To End

3.1 Composite NLP score (per email)

```
total_nlp_score = 0.45 * transformer_phishing_probability
                + 0.55 * tfidf_logreg_phishing_probability
```

If the transformer is disabled/unavailable, total_nlp_score falls back to the TF-IDF probability alone.

3.2 Content-step score (0-100)

```
content_score = 0.65 * (total_nlp_score * 100)
              + 0.35 * regex_cue_score      # urgency / financial / authority phrase hits
```

Status thresholds: score ≥ 50 → red (problem detected); 0 < score < 50 → amber (needs review); score = 0 → green.

3.3 Overall fraud score (fusion.py — unchanged)

```
fraud_score = 0.35*auth_score + 0.20*origin_score
            + 0.25*content_score + 0.20*links_score
(overridden to 100 on a hard-fail: DMARC hard-reject + failed alignment,
 or a malicious attachment hash match)
```

Verdict: **Fraudulent** if fraud_score ≥ FRAUD_THRESHOLD (default 70), **Suspicious** if ≥ FLAG_THRESHOLD (default 40), otherwise **Clean**. Both thresholds are tunable in `.env`.

4. Project Layout

```
mailguard/
  README.md
  docker-compose.yml
  GUIDE.pdf                       <- this file
  backend/
    requirements.txt
    .env.example
    generate_secrets.py
    seed_demo.py
    data/
      email_text.csv              <- training dataset (53.6k rows)
    sample_emails/
      phishing_sample1.eml        <- renamed "Spot Admissions" email
      legit_sample1.eml           <- renamed "Two new free courses" email
      clean_newsletter.eml
      suspicious_payroll_update.eml
    app/
      main.py                     <- FastAPI app, routes, /health/nlp
      config.py                   <- env-driven settings (incl. USE_TRANSFORMER_MODEL)
      pipeline.py                 <- orchestrates the 5 steps
      email_parser.py
      modules/
        nlp_engine.py             <- NEW: transformer + TF-IDF/LR + composite engine
        content_module.py         <- REWRITTEN: calls nlp_engine, builds flag_summary
        auth_module.py
        origin_module.py
        link_module.py
        fusion.py                 <- composite fraud-score engine (unchanged)
      model_cache/                <- auto-created: cached trained TF-IDF/LR model
      routers/, models.py, schemas.py, database.py, auth.py
  frontend/
    src/
      components/
        FlowChart.jsx             <- UPDATED: shows the "Why this was flagged" panel
        Dashboard.jsx, EmailDetail.jsx, Login.jsx, OriginMap.jsx, RiskBadge.jsx
      styles.css                  <- UPDATED: .nlp-flag-summary styling
      App.jsx, api.js, main.jsx
```

5. How To Run It On Your System

Two paths are supported. Pick whichever you already have installed.

5.1 Fastest path: Docker

Requires Docker Desktop installed and running.

```
cd mailguard
cp backend/.env.example backend/.env
python3 backend/generate_secrets.py
# copy the two printed values (SECRET_KEY and ANALYST_PASSWORD_HASH)
# into backend/.env

docker compose up --build
```

Frontend: http://localhost:5173 · Backend docs: http://localhost:8000/docs

5.2 Manual path (no Docker) — recommended if you're short on time before a demo

Step 1 — Prerequisites

- Python 3.10+ (check: `python3 --version`)
- Node.js 18+ and npm (check: `node --version`)

Step 2 — Backend setup

```
cd mailguard/backend
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

pip install -r requirements.txt
# NOTE: transformers + torch are ~1-2 GB combined and the phishing
# transformer model itself is ~250 MB, downloaded on first run.
# If you are short on time/bandwidth before your demo, install
# everything EXCEPT those two lines and set USE_TRANSFORMER_MODEL=false
# in your .env (see Step 3) — the app still works fully on TF-IDF + regex.

cp .env.example .env
python3 generate_secrets.py
# paste the two printed values into .env (SECRET_KEY, ANALYST_PASSWORD_HASH)
```

Step 3 — Configure .env

Open `backend/.env` and set at minimum:

```
SECRET_KEY=<paste from generate_secrets.py>
ANALYST_USERNAME=analyst
ANALYST_PASSWORD_HASH=<paste from generate_secrets.py>
USE_TRANSFORMER_MODEL=true      # set to false for a faster/offline demo
```

Step 4 — Start the backend

```
uvicorn app.main:app --reload --port 8000
```

First startup will take longer than usual: it trains the TF-IDF/Logistic-Regression model from `data/email_text.csv` (roughly 5-10 seconds) and, if enabled, downloads the transformer model. Every startup after that is fast because both are cached.

> Right after startup, open `http://localhost:8000/health/nlp` in a browser. It reports whether the TF-IDF model trained and whether the transformer loaded — confirm this BEFORE your demo, not during it.

Step 5 — Start the frontend (new terminal)

```
cd mailguard/frontend
npm install
npm run dev
```

Open http://localhost:5173, log in with the analyst username/password you chose in Step 3.

Step 6 — Load demo data (optional but recommended)

With the backend running, from a third terminal:

```
cd mailguard/backend
python3 seed_demo.py <your_analyst_username> <your_analyst_password>
```

This uploads all four sample .eml files (including the newly added `phishing_sample1.eml` and `legit_sample1.eml`) so the dashboard has something to show immediately. You can also just drag-and-drop any .eml file onto the dashboard's upload area instead.

6. Suggested Demo Walkthrough (for judges)

1. Upload `phishing_sample1.eml` and let it score.
2. Click through the 5-step flowchart, ending on 'NLP Semantics & Social Engineering Analysis'.
3. Point out the highlighted **'Why this was flagged'** box — read the model's own explanation aloud.
4. Open 'Full evidence' below it and show the transformer's raw probabilities alongside the TF-IDF model's probabilities and its top contributing words — this demonstrates two independently-reasoned models agreeing (or disagreeing) explainably, not a black box.
5. Upload `legit_sample1.eml` for contrast and repeat, to show a lower score and a different explanation.
6. Mention the fusion score: it isn't NLP alone — auth, origin, and link checks all contribute too.

7. Known Limitations Found During Integration

Being upfront about these strengthens a hackathon Q&A far more than hiding them:

- **Dataset quirk:** a handful of rows near the end of `email_text.csv` have unescaped commas inside the text field, which breaks strict CSV parsing. `nlp_engine.py` loads the file with `on_bad_lines='skip'` so training doesn't crash — a handful of rows out of 53,672 are dropped. Worth cleaning at the source if you keep developing this.
- **TF-IDF false positives on short, benign, corporate-style emails:** tested standalone (transformer off), the TF-IDF/Logistic-Regression model — trained mostly on classic marketing/scam-style spam plus a handful of real Google/Supabase security-notice emails — sometimes over-scores short, clean, professional messages (e.g. a one-line meeting agenda) because it hasn't seen much of that style labelled 'legitimate'. The transformer component is meant to counterbalance this by recognizing broader phrasing patterns — **test with** `USE_TRANSFORMER_MODEL=true` **before your demo** to confirm the blended score behaves as expected on your own test emails.
- **Model accuracy is dataset-bound:** the TF-IDF/LR model reported ~98.8% accuracy on a held-out 30% split of email_text.csv — a strong number, but it reflects this dataset's specific vocabulary (largely older-style spam/phishing wording), not necessarily every modern phishing style.
- **Transformer download size:** ~250 MB on first run, plus the torch/transformers packages (~1-2 GB). Budget time for this before a live demo, or set `USE_TRANSFORMER_MODEL=false` to skip it entirely.

8. Suggested Improvements

8.1 Model quality

- Add a set of modern, benign corporate/personal emails (newsletters, course notifications, calendar invites) to the training data, labelled 'legitimate', to correct the false-positive tendency noted above.
- Calibrate the 0.45/0.55 blend weight and the FLAG_THRESHOLD/FRAUD_THRESHOLD values against a small, held-out set of real emails you personally know the ground truth for, rather than relying only on the dataset's own train/test split.

- Log every prediction (model scores + final verdict + analyst's eventual manual verdict, once that exists) so you can compute real precision/recall over time and retrain periodically.
- Consider a lightweight explainability layer for the transformer output too (e.g. attention-based or gradient-based word importance), to match the TF-IDF model's per-word explanation.

8.2 Data & robustness

- Clean the malformed rows in email_text.csv at the source instead of skipping them at load time.
- Cap or chunk very long email bodies more carefully (currently truncated at 4000 characters before scoring) — consider summarizing long emails first rather than simple truncation.
- Add a scheduled/CI job that retrains and re-validates the TF-IDF model whenever the dataset changes, rather than relying on the on-disk cache going stale silently.

8.3 Product & UX

- Show a small trend/history view per sender or domain, not just per email, so an analyst can spot a sender that's been creeping up in risk score over multiple messages.
- Let the analyst give feedback on a verdict (confirm/override) directly in the UI, feeding back into future model retraining (active learning).
- Add a bulk-upload endpoint (multiple .eml files at once) — currently one at a time.

8.4 Security & ops (for beyond-hackathon use)

- Put the whole stack behind HTTPS via a reverse proxy (Caddy/nginx + Let's Encrypt) — the README already flags this but it's easy to forget on demo day.
- Move from SQLite to Postgres if this needs to run continuously with concurrent uploads.
- Rate-limit the /emails/upload endpoint — a malicious upload flood could exhaust the transformer's inference time budget.
- Add multi-analyst accounts and audit logging (who reviewed which email, when) instead of the single shared analyst login.

9. Troubleshooting Quick Reference

Symptom	Likely cause / fix
Backend refuses to start, prints a config error	SECRET_KEY or ANALYST_PASSWORD_HASH missing in .env — rerun generate_secrets.py and paste both values in.
Training the TF-IDF model fails on startup	Confirm backend/data/email_text.csv exists and has 'label,text' columns. Delete backend/app/model_cache/ to force retraining if you changed the CSV.
Transformer never loads / /health/nlp shows transformer_ready: false	No internet on first run, or USE_TRANSFORMER_MODEL=false in .env. The app still works on TF-IDF + regex alone — this is a soft-fail by design, not a bug.

Frontend can't reach backend (CORS/network error)	Confirm FRONTEND_ORIGIN in backend/.env matches the URL you're loading the frontend from (default http://localhost:5173).
seed_demo.py fails to upload one of the sample emails	That exact file was probably already ingested earlier (the API rejects duplicate SHA-256 hashes) — delete it from the dashboard first, or ignore, it's expected on a second run.